

ICT2216:

Secure Software Development

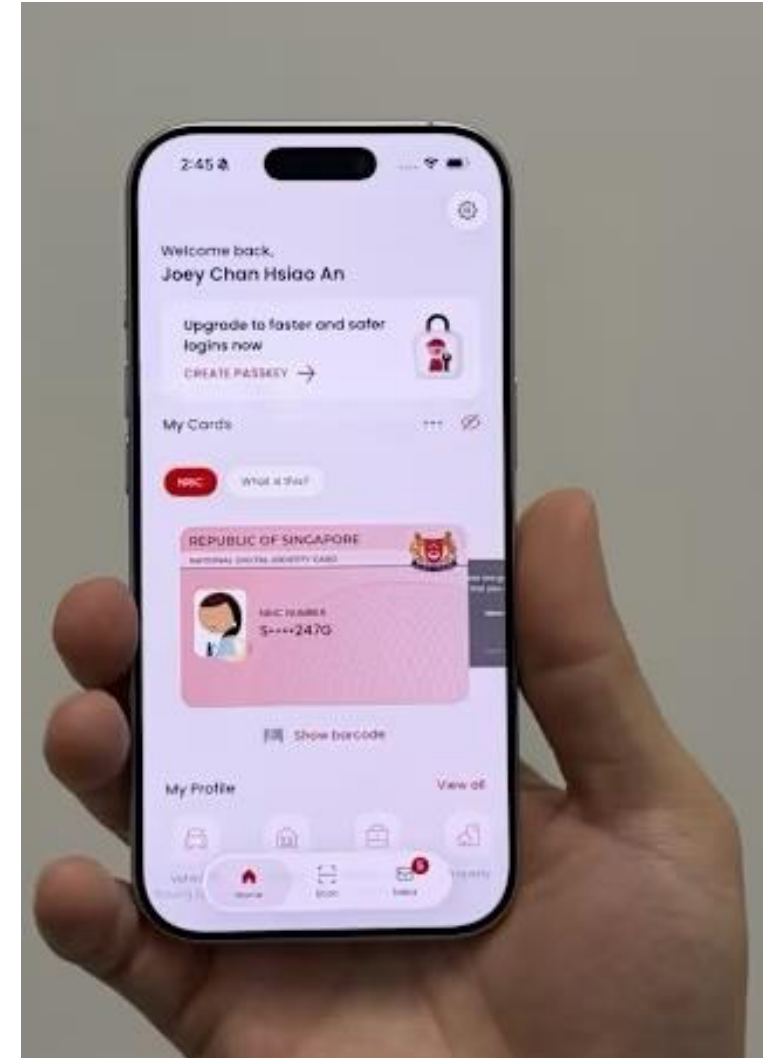
Secure Software Implementation

Part 3: Test Automation and References for Security Test



Singpass to roll out passkey logins from Jul 1

- Singpass users will be able to log in with passkeys from Wednesday (Jul 1), offering stronger protection against phishing and other cyberthreats.
- The Government Technology Agency of Singapore (GovTech Singapore) announced the rollout on Tuesday (Jun 30), saying passkeys provide an additional security layer while maintaining a smooth login experience.
- The move comes as phishing remained Singapore's second most common scam last year, with losses of almost S\$40 million (US\$30.8 million).
- Unlike passwords, which can be stolen, or QR codes, which scammers can trick users into scanning, passkeys work by verifying a private key stored on the user's device against a public key registered in the Singpass system.



Singpass to roll out passkey logins from Jul 1

Because no passwords or one-time passwords are entered or transmitted during login, there is nothing for scammers to intercept – even if a user lands on a fraudulent website.

Passkeys will be added to existing authentication methods such as QR login, face verification and SMS one-time passwords.

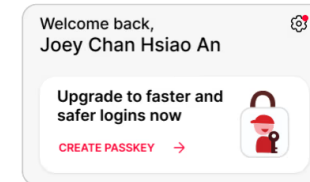
If a device is lost or stolen, the Singpass app and passkey on it will be automatically deactivated when the user sets up Singpass on another device.

The initial beta phase is available to iPhone (iOS) users, and Android support will follow in subsequent phases. Users who have enabled notifications will receive a push notification in the Singpass app when the feature becomes available to them.

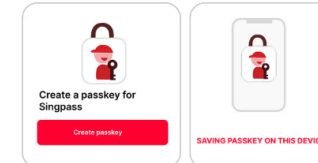
How to set up and use a Singpass passkey

SETTING UP

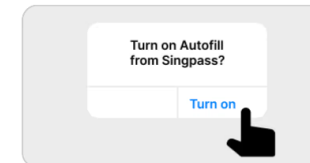
- 1 Open Singpass app and tap on Create Passkey



- 2 Passkey creation in progress



- 3 Turn on Autofill for Singpass when prompted

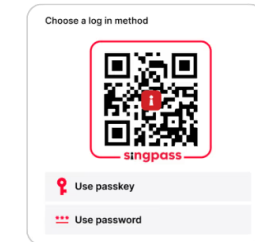


- 4 Passkey created successfully

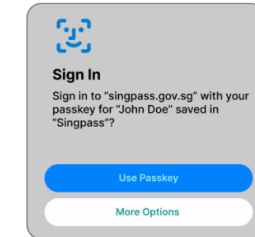


LOGGING IN

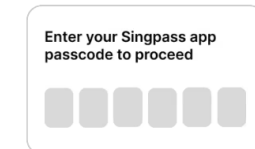
- 1 Tap on use passkey to log in



- 2 Verify using FaceID or fingerprint



If biometrics not set up, use passcode



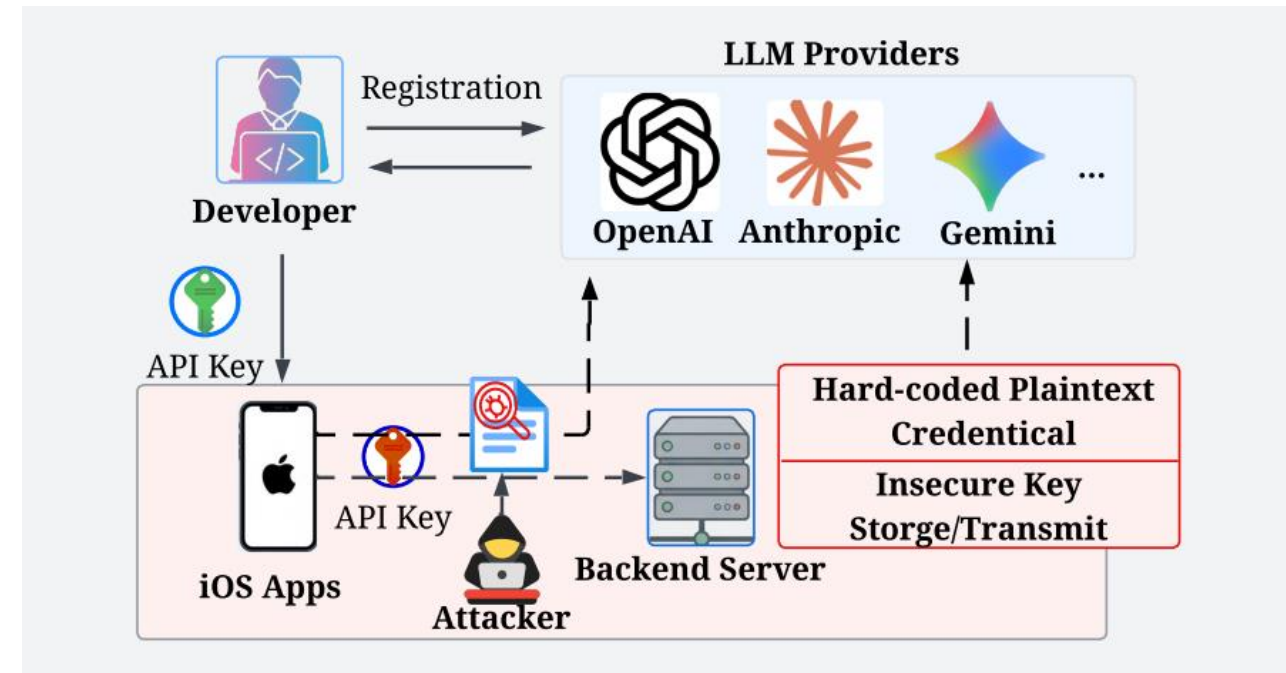
- 3 Logged in successfully

282 iOS AI Apps Leak API Keys and Open AI Proxy Access in Network Traffic Study

SIT Internal

All 282 fell into one of three groups:

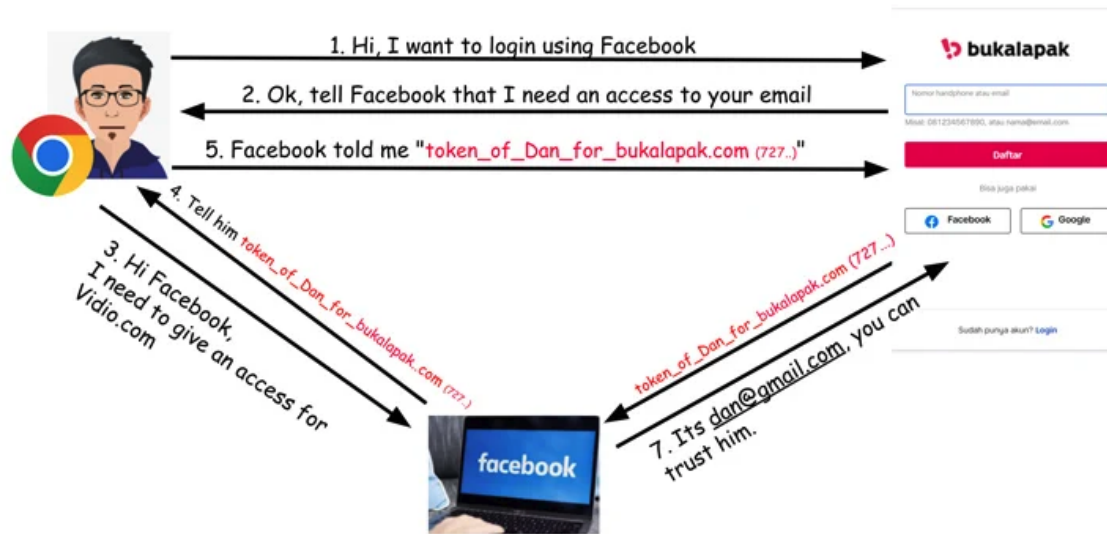
- **Plaintext keys (54 apps):** the key is sent in the open, readable from a single captured request.
- **No key needed (92 apps):** the app routes requests through a server that answers anyone, with no check on who is asking. An open relay to a paid AI account.
- **Replayable tokens (136 apps, the most common):** the app hands out temporary access tokens instead of the raw key, the approach that is supposed to be safer, but the tokens leak in the same traffic and were usually still valid when captured. Some were not temporary at all, as the cases below show.



By now, do you know how to secure the API key?

Critical OAuth Flaws Uncovered in Grammarly, Vidio, and Bukalapak Platforms

Oct 25, 2023 Newsroom



Critical security flaws have been disclosed in the Open Authorization (OAuth) implementation of popular online services such as Grammarly, Vidio, and Bukalapak, building upon previous shortcomings uncovered in [Booking.com](#) and [Expo](#).

The weaknesses, now addressed by the respective companies following responsible disclosure between February and April 2023, could have allowed malicious actors to obtain access tokens and potentially hijack user accounts.

OAuth is a [standard](#) that's commonly used as a mechanism for cross-application access, granting websites or applications access to their information on other websites, such as Facebook, but without giving them the passwords.



Trending News



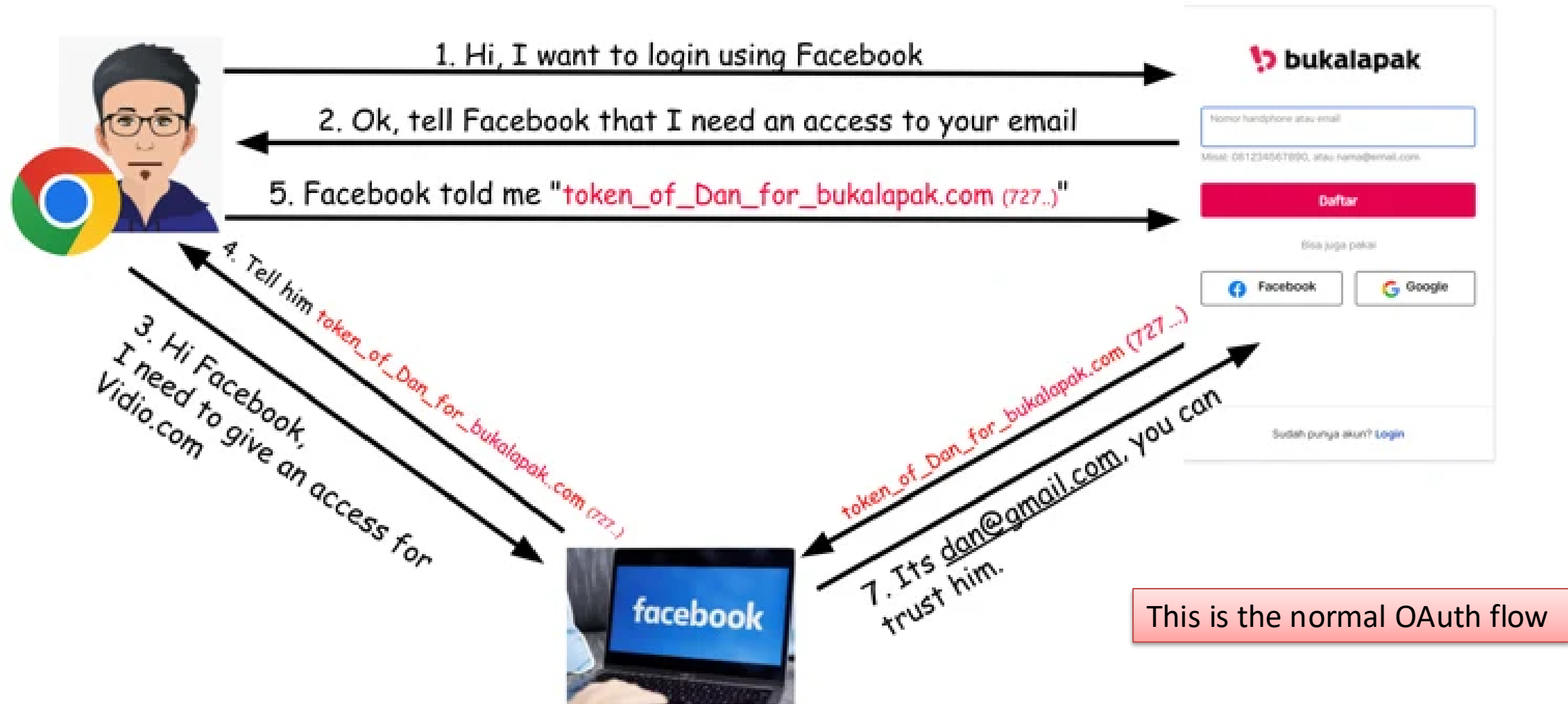
Okta's Support System Breach Exposes Customer Data to Unidentified Threat Actors



Cisco Zero-Day Exploited to Implant Malicious Lua Backdoor

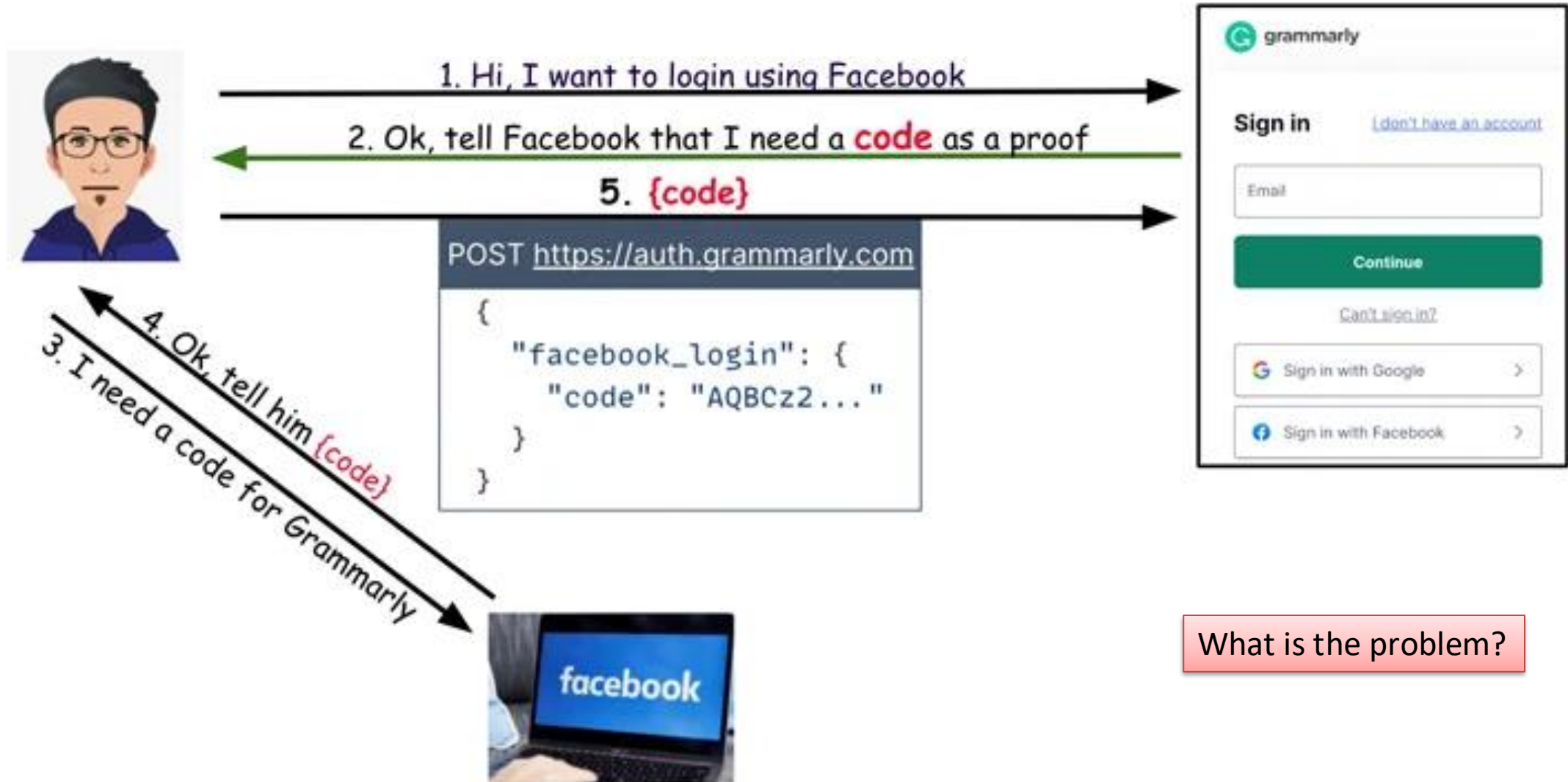
Critical OAuth Flaws Uncovered in Grammarly, Vidio, and Bukalapak Platforms

SIT Internal



Critical OAuth Flaws Uncovered in Grammarly, Vidio, and Bukalapak Platforms

SIT Internal



What is the problem?

Testing AI if it is secure?

US issues executive order on safe, secure, and trustworthy artificial intelligence

The new executive order acknowledges that as AI's capabilities grow, its implications for Americans' safety and security also grows



Nidhi Singal

Updated Oct 31, 2023, 9:31 AM IST



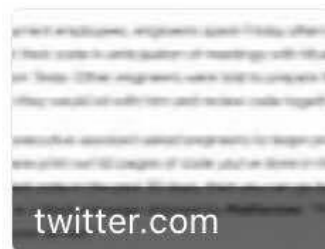
America acknowledges that as AI's capabilities grow, its implications for Americans' safety and security also grows

Print your code for review?



 r/programming
u/DisneyLegalTeam • 17h

Twitter engineers asked to print out their last 30-60 days of code. And to be ready for pairings with Musk



Elon Code Review

Features

Testimonials

Pricing

Sign in

Print Your Code for review by Elon Musk.

Most code is reviewed in Github, Gitlab or somewhere online, but that is *too* efficient. Why not **print** out your code and submit the printout by **fax** for review by **Elon Musk** personally!

Get 6 months free

▶ Watch video

No fuss way to review your code.



Print Your Code

Go to your favorite text editor, select all the code you wrote in the last 30 days ago and copy paste it into Microsoft Word. Then press Print. Easy as that.



Fax Your Printouts

Take your code print outs and fax them to our toll free number. Our AI will recombine the pages in correct order and then robot technology will deliver them to Elon for review.



Wait for the mailman

Get your code review print out back with Elon Musk's handwritten notes and hand drawn emojis.



Send your code to space

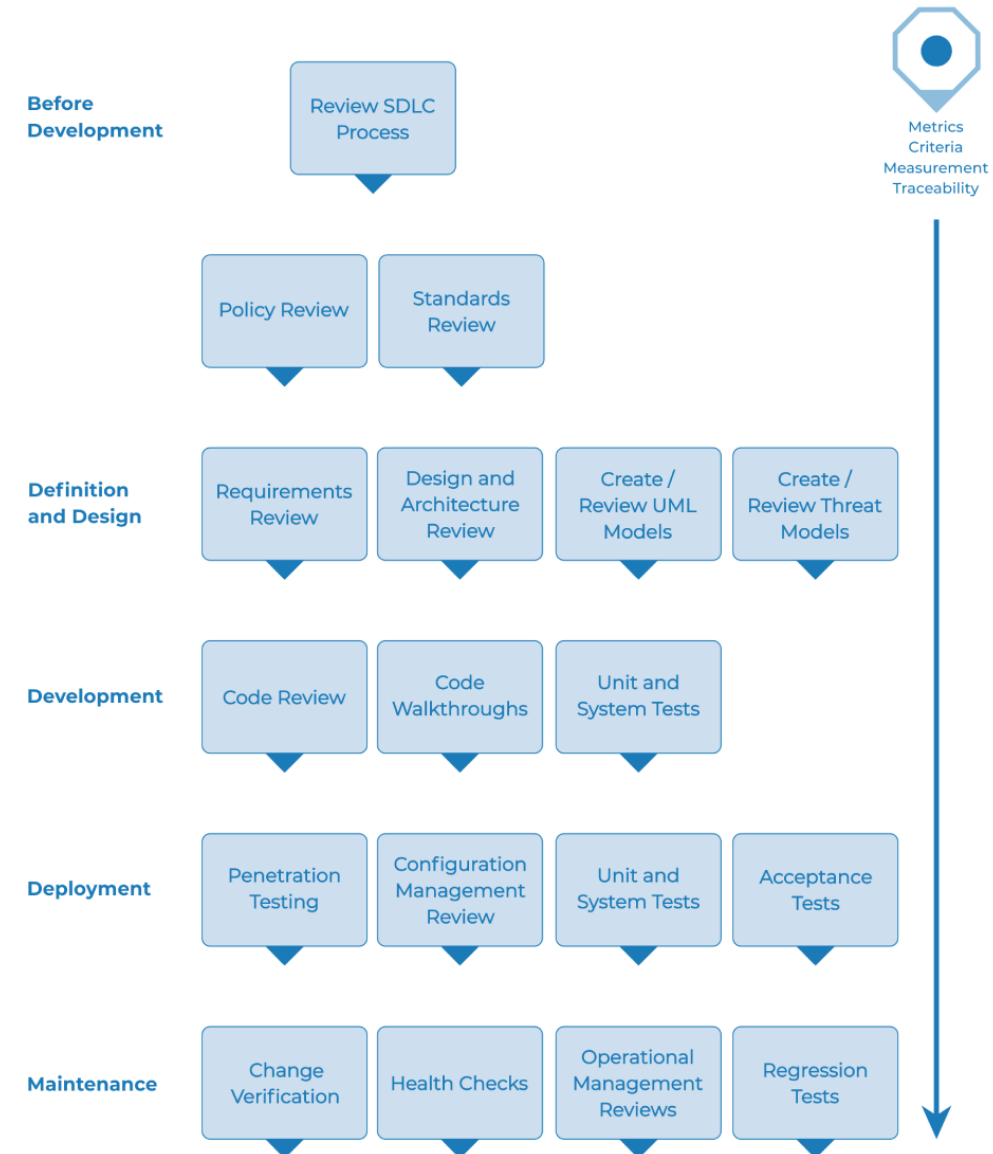
All code reviews merged after a review by Elon Musk are automatically re-printed and sent to space aboard a SpaceX rocket. *Enterprise plan only.

Agenda

- Test Automation
 - Unit Test, Integration Test, GUI Test
- Security Test References
 - OWASP Application Security Verification Standard (ASVS)
<https://owasp.org/www-project-application-security-verification-standard/>
 - OWASP Web Security Testing Guide
<https://owasp.org/www-project-web-security-testing-guide/v42/>

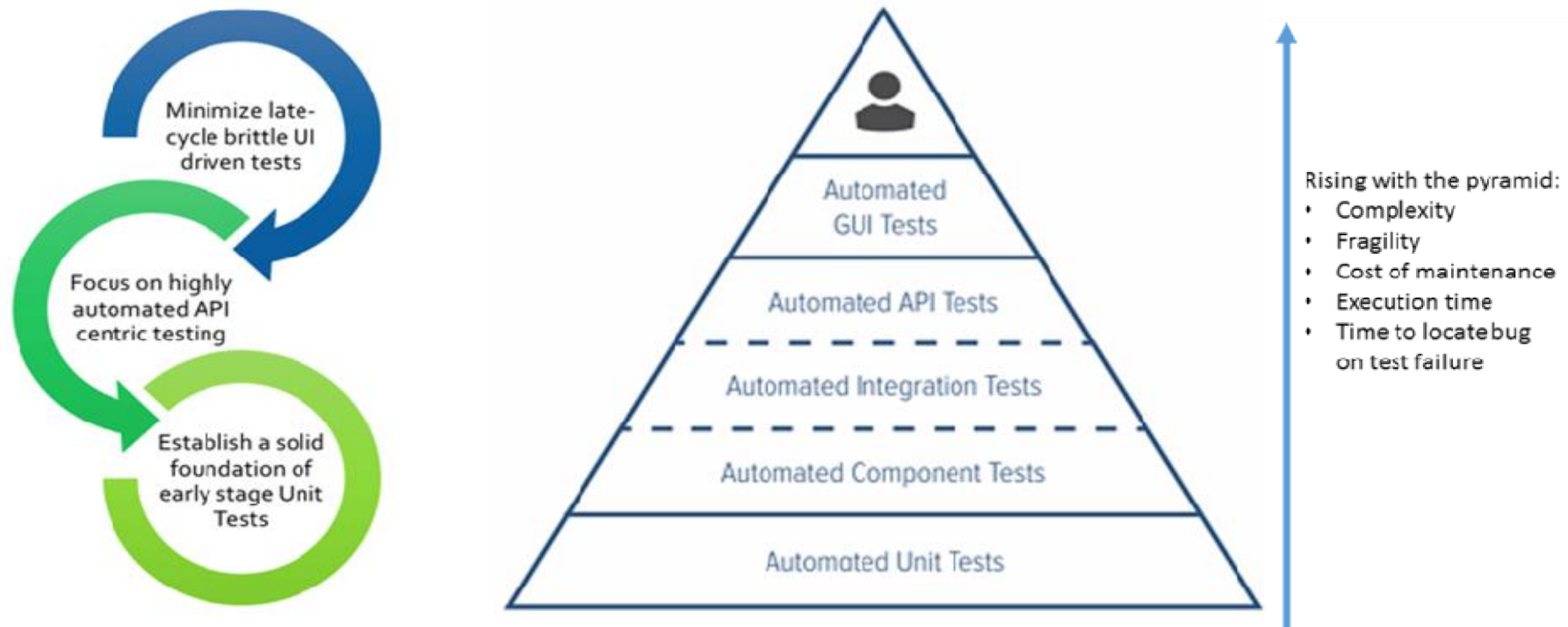
OWASP Testing Framework in OWASP Testing Guide

- It includes security tests throughout the development process, and to **test early** and **test often** instead of waiting until code is completely built which can be costly.



OWASP Testing Framework in OWASP Testing Guide

- Commonly used **testing pyramid model** – ideal distribution of testing resources, with automated unit tests forming the bulk of your tests.



Some **automated testing tools** which you may try in team project:



Test Automation

- Unit test is to test individual units or components of an application, e.g. a single method or a single class, to validate that each is working properly as intended.

```
class UserTest < ActiveSupport::TestCase

  def setup
    @user = User.new(name: 'Example User', email: 'user@example.com',
                     password: 'foobar', password_confirmation: 'foobar')
  end

  test 'should be valid' do
    assert @user.valid?
  end

  test 'name should be present' do
    @user.name = ' '
    assert_not @user.valid?
  end
end
```

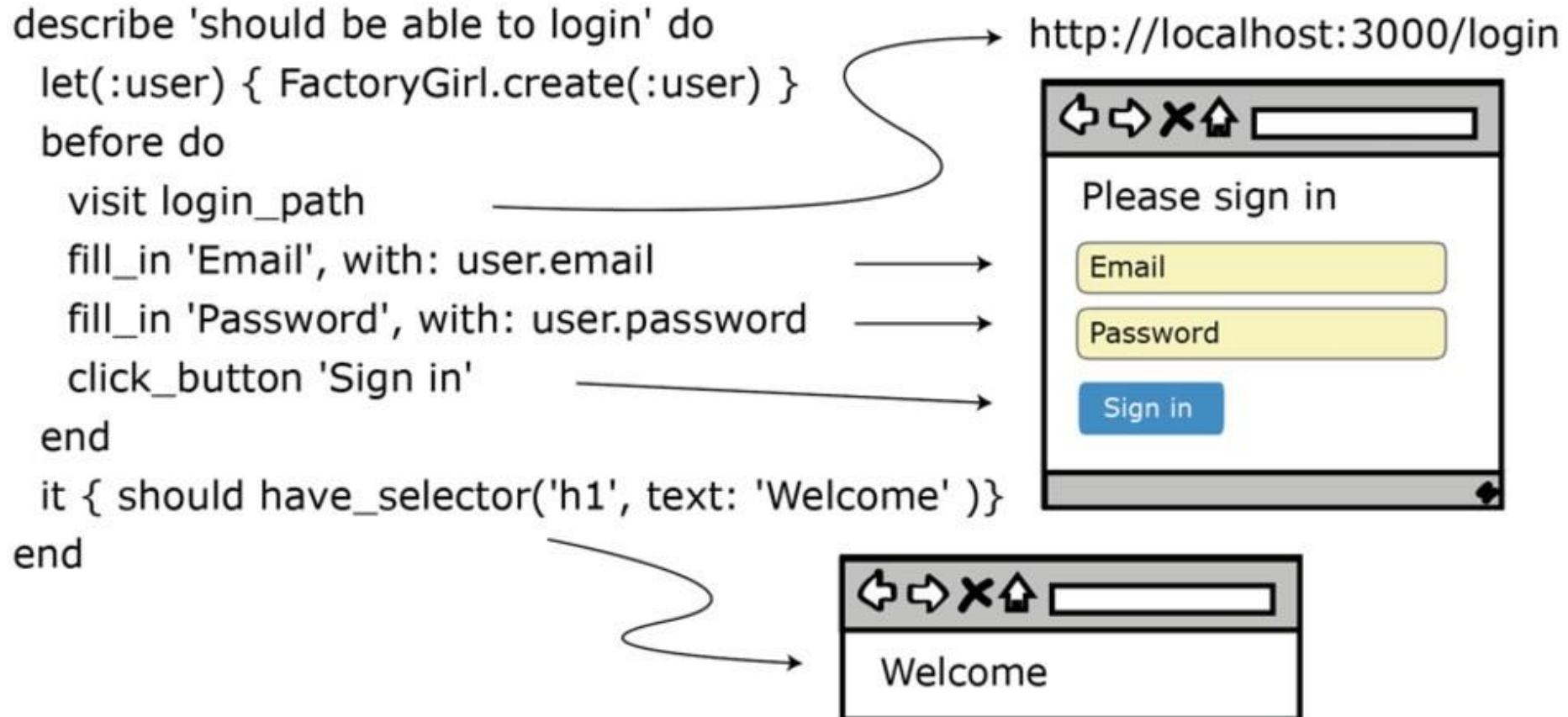


Required fields
Password length
Name length
Valid email address
Unique email address



Test Automation

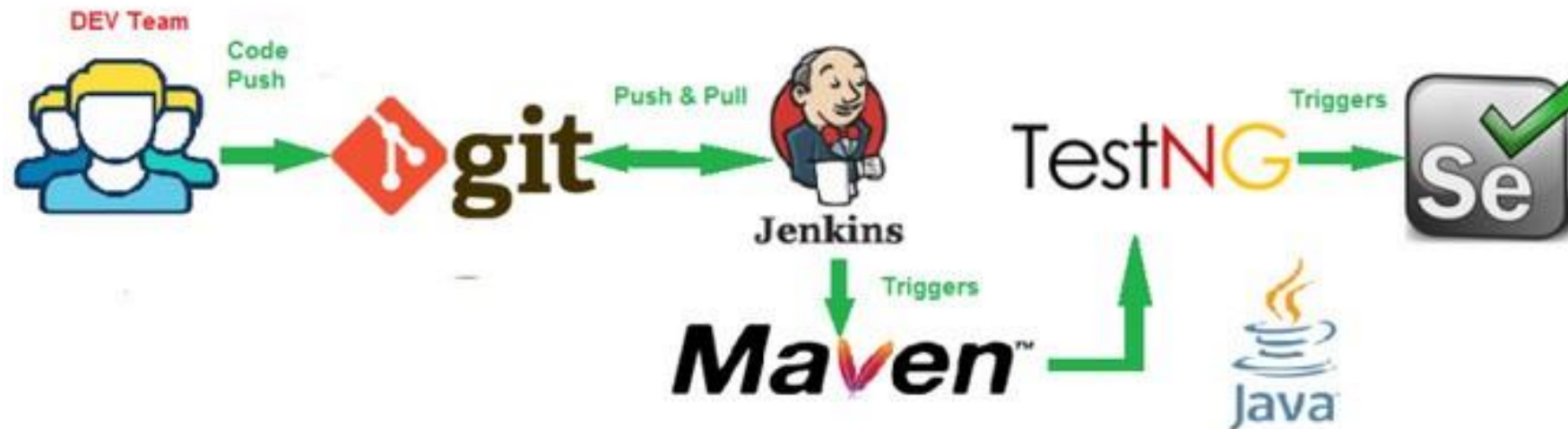
- Integration test, system test and UI test are to validate the behaviour of multiple units together, or the application as a whole.



Test Automation

- An example of performing automated integration and UI tests in Jenkins:

How Run Selenium Tests in Jenkins Using Maven



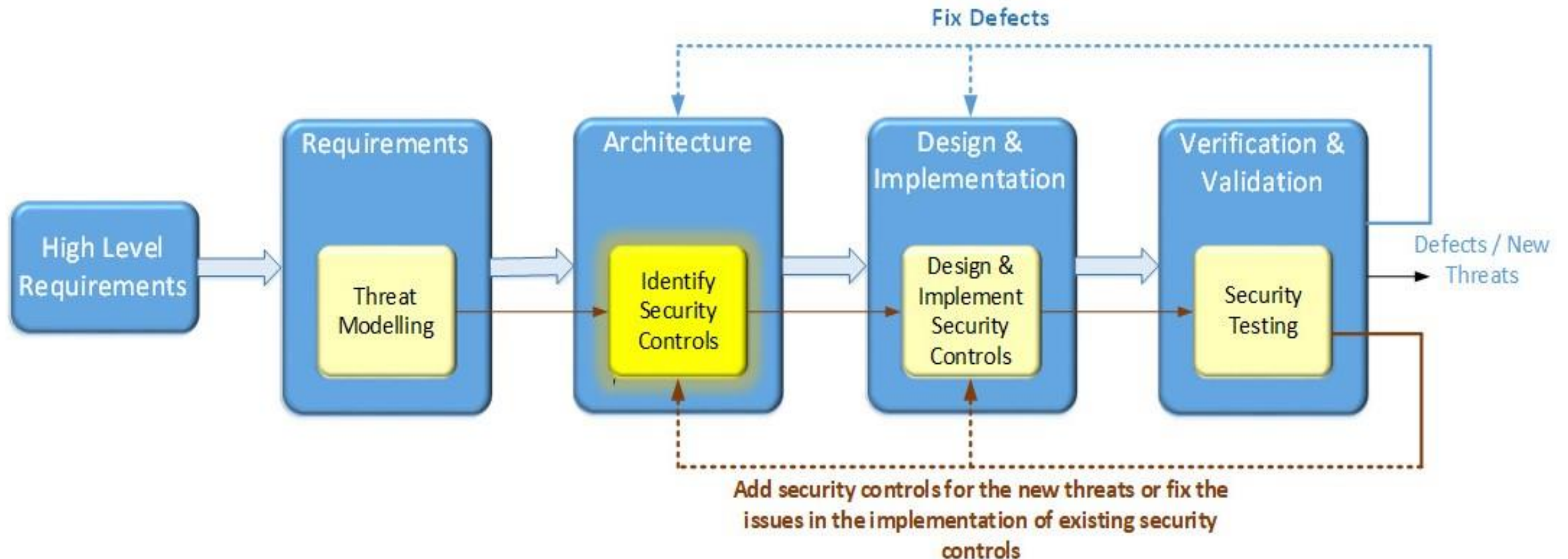
Maven: tool for building and managing Java-based project

TestNG: test automation framework which will be invoked by maven- surefire-plugin to run the tests under the annotation @Test)

Selenium WebDriver: library for automation of browser interaction

Test Automation

The main objective of **security testing** is to verify that the implemented **security controls** operate as expected meeting the **security requirements**.



OWASP Application Security Verification Standard (ASVS)

- OWASP has published useful testing recommendations which can help in performing security tests.

OWASP ASVS – a catalog of **verification requirements** for the **security controls** and **security requirements**.



- Security testing is never an exact science where all possible test cases can be fully defined.
- Not all security tests can be automated and **balanced** approach of both **automated and manual testing** are required.

OWASP Application Security Verification Standard (ASVS)

- OWASP Application Security Verification Standard (ASVS) organizes verification requirements into 14 broad categories:

V1: Architecture, Design and Threat Modeling Requirements

V2: Authentication Verification Requirements

V3: Session Management Verification Requirements

V4: Access Control Verification Requirements

V5: Validation, Sanitization and Encoding Verification Requirements

V6: Stored Cryptography Verification Requirements

V7: Error Handling and Logging Verification Requirements

V8: Data Protection Verification Requirements

V9: Communications Verification Requirements

V10: Malicious Code Verification Requirements

V11: Business Logic Verification Requirements

V12: File and Resources Verification Requirements

V13: API and Web Service Verification Requirements

V14: Configuration Verification Requirements

Application Security Verification Levels

The Application Security Verification Standard defines three security verification levels, with each level increasing in depth.

- ASVS Level 1 is for low assurance levels, and is completely penetration testable
- ASVS Level 2 is for applications that contain sensitive data, which requires protection and is the recommended level for most apps
- ASVS Level 3 is for the most critical applications - applications that perform high value transactions, contain sensitive medical data, or any application that requires the highest level of trust.

	Applicability	Building			Building, Configuration, Deployment Assurance and Verification			Assurance and Verification	
Level 1	All apps		Secure Coding	Standards and checklists	Secure & Peer Code Review	DevSecOps	Unit and Integration Tests	Penetration Testing	DAST
Level 2	All apps	Security Architecture and Reviews	Secure Coding	Standards and checklists	Secure & Peer Code Review	DevSecOps	Unit and Integration Tests	Hybrid Reviews	SAST
Level 3	High Assurance	Security Architecture and Reviews	Secure Coding	Standards and checklists	Secure & Peer Code Review	DevSecOps	Unit and Integration Tests	Hybrid Reviews	SAST
Legend		Acceptable	Suitable						

OWASP Application Security Verification Standard (ASVS)

- Each category of OWASP ASVS contains a collection of related verification requirements; e.g. V2: Authentication Verification Requirements as follows:
 - V2.1: [Password Security Requirements](#)
 - V2.2: General Authenticator Requirements
 - V2.3: Authenticator Lifecycle Requirements
 - V2.4: [Credential Storage Requirements](#)
 - V2.5: [Credential Recovery Requirements](#)
 - V2.6: Look-up Secret Verifier Requirements
 - V2.7: Out of Band Verifier Requirements
 - V2.8: Single or Multi Factor One Time Verifier Requirements
 - V2.9: Cryptographic Software and Devices Verifier Requirements V2.10: Service Authentication Requirements

References:

- ...
- [OWASP Web Security Testing Guide: Testing for Authentication](#)
- OWASP Cheat Sheet ...

Password Security Requirements

#	Description	L1	L2	L3	CWE	NIST §
2.1.1	Verify that user set passwords are at least 12 characters in length (after multiple spaces are combined). (C6)	✓	✓	✓	521	5.1.1.2
2.1.2	Verify that passwords of at least 64 characters are permitted, and that passwords of more than 128 characters are denied. (C6)	✓	✓	✓	521	5.1.1.2
2.1.3	Verify that password truncation is not performed. However, consecutive multiple spaces may be replaced by a single space. (C6)	✓	✓	✓	521	5.1.1.2
2.1.4	Verify that any printable Unicode character, including language neutral characters such as spaces and Emojis are permitted in passwords.	✓	✓	✓	521	5.1.1.2
2.1.5	Verify users can change their password.	✓	✓	✓	620	5.1.1.2
2.1.6	Verify that password change functionality requires the user's current and new password.	✓	✓	✓	620	5.1.1.2

Password Security Requirements

#	Description	L1	L2	L3	CWE	NIST §
2.1.7	Verify that passwords submitted during account registration, login, and password change are checked against a set of breached passwords either locally (such as the top 1,000 or 10,000 most common passwords which match the system's password policy) or using an external API. If using an API a zero knowledge proof or other mechanism should be used to ensure that the plain text password is not sent or used in verifying the breach status of the password. If the password is breached, the application must require the user to set a new non-breached password. (C6)	✓	✓	✓	521	5.1.1.2
2.1.8	Verify that a password strength meter is provided to help users set a stronger password.	✓	✓	✓	521	5.1.1.2
2.1.9	Verify that there are no password composition rules limiting the type of characters permitted. There should be no requirement for upper or lower case or numbers or special characters. (C6)	✓	✓	✓	521	5.1.1.2
2.1.10	Verify that there are no periodic credential rotation or password history requirements.	✓	✓	✓	263	5.1.1.2
2.1.11	Verify that "paste" functionality, browser password helpers, and external password managers are permitted.	✓	✓	✓	521	5.1.1.2
2.1.12	Verify that the user can choose to either temporarily view the entire masked password, or temporarily view the last typed character of the password on platforms that do not have this as built-in functionality.	✓	✓	✓	521	5.1.1.2

Credential Storage Requirements

#	Description	L1	L2	L3	CWE	NIST §
2.4.1	Verify that passwords are stored in a form that is resistant to offline attacks. Passwords SHALL be salted and hashed using an approved one-way key derivation or password hashing function. Key derivation and password hashing functions take a password, a salt, and a cost factor as inputs when generating a password hash. (C6)		✓	✓	916	5.1.1.2
2.4.2	Verify that the salt is at least 32 bits in length and be chosen arbitrarily to minimize salt value collisions among stored hashes. For each credential, a unique salt value and the resulting hash SHALL be stored. (C6)		✓	✓	916	5.1.1.2
2.4.3	Verify that if PBKDF2 is used, the iteration count SHOULD be as large as verification server performance will allow, typically at least 100,000 iterations. (C6)		✓	✓	916	5.1.1.2
2.4.4	Verify that if bcrypt is used, the work factor SHOULD be as large as verification server performance will allow, with a minimum of 10. (C6)		✓	✓	916	5.1.1.2
2.4.5	Verify that an additional iteration of a key derivation function is performed, using a salt value that is secret and known only to the verifier. Generate the salt value using an approved random bit generator [SP 800-90Ar1] and provide at least the minimum security strength specified in the latest revision of SP 800-131A. The secret salt value SHALL be stored separately from the hashed passwords (e.g., in a specialized device like a hardware security module).		✓	✓	916	5.1.1.2

Credential Recovery Requirements

#	Description	L1	L2	L3	CWE	NIST §
2.5.1	Verify that a system generated initial activation or recovery secret is not sent in clear text to the user. (C6)	✓	✓	✓	640	5.1.1.2
2.5.2	Verify password hints or knowledge-based authentication (so-called "secret questions") are not present.	✓	✓	✓	640	5.1.1.2
2.5.3	Verify password credential recovery does not reveal the current password in any way. (C6)	✓	✓	✓	640	5.1.1.2
2.5.4	Verify shared or default accounts are not present (e.g. "root", "admin", or "sa").	✓	✓	✓	16	5.1.1.2 / A.3
2.5.5	Verify that if an authentication factor is changed or replaced, that the user is notified of this event.	✓	✓	✓	304	6.1.2.3
2.5.6	Verify forgotten password, and other recovery paths use a secure recovery mechanism, such as time-based OTP (TOTP) or other soft token, mobile push, or another offline recovery mechanism. (C6)	✓	✓	✓	640	5.1.1.2
2.5.7	Verify that if OTP or multi-factor authentication factors are lost, that evidence of identity proofing is performed at the same level as during enrollment.		✓	✓	308	6.1.2.3

V10 Malicious Code

#	Description	L1	L2	L3	CWE
10.1.1	Verify that a code analysis tool is in use that can detect potentially malicious code, such as time functions, unsafe file operations and network connections.			✓	749
#	Description	L1	L2	L3	CWE
10.2.1	Verify that the application source code and third party libraries do not contain unauthorized phone home or data collection capabilities. Where such functionality exists, obtain the user's permission for it to operate before collecting any data.		✓	✓	359
10.2.2	Verify that the application does not ask for unnecessary or excessive permissions to privacy related features or sensors, such as contacts, cameras, microphones, or location.		✓	✓	272
10.2.3	Verify that the application source code and third party libraries do not contain back doors, such as hard-coded or additional undocumented accounts or keys, code obfuscation, undocumented binary blobs, rootkits, or anti-debugging, insecure debugging features, or otherwise out of date, insecure, or hidden functionality that could be used maliciously if discovered.			✓	507
10.2.4	Verify that the application source code and third party libraries do not contain time bombs by searching for date and time related functions.			✓	511

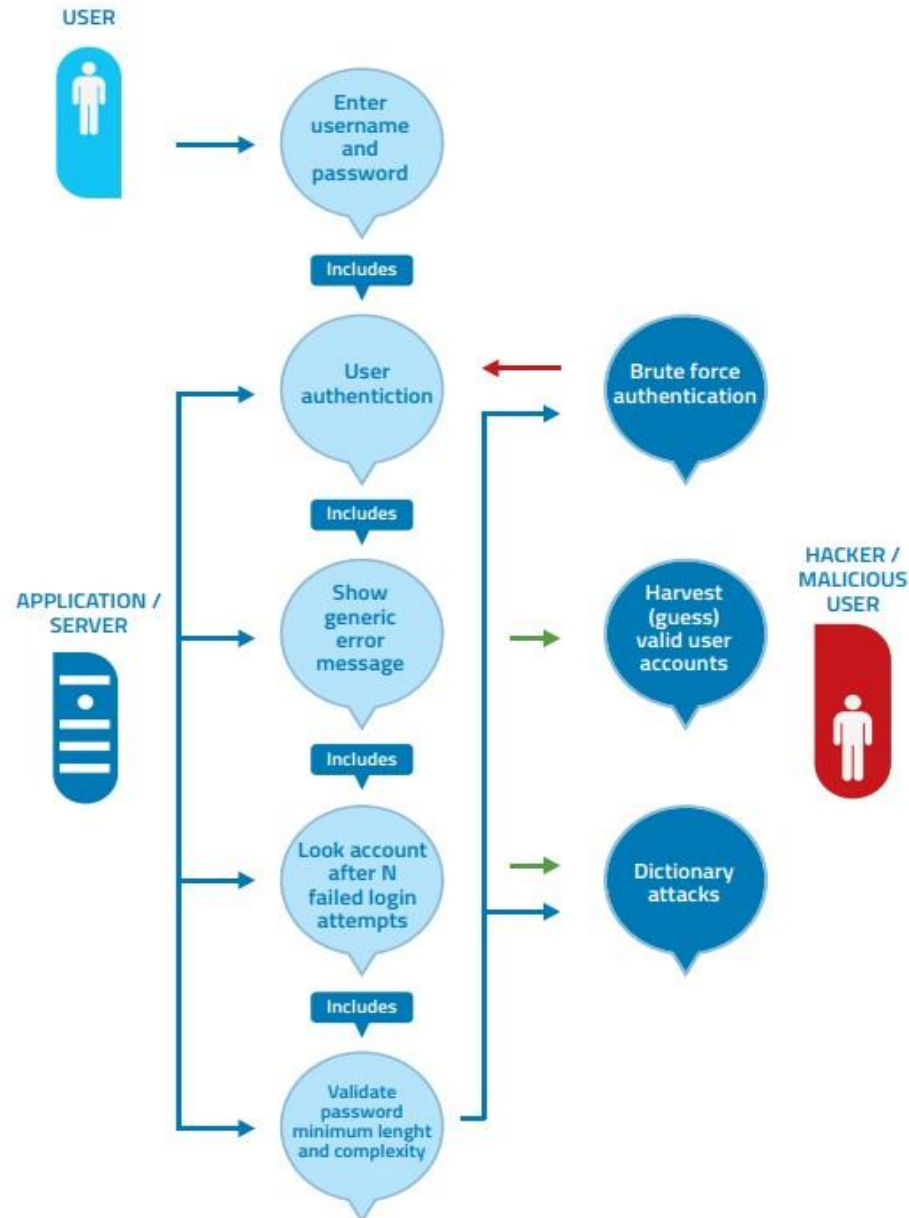
OWASP Web Security Testing Guide

OWASP Web Security Testing Guide

- It provides additional test cases on security testing which is organized into 11 test categories as follows:

- | | |
|--|--|
| <ul style="list-style-type: none">• Information Gathering (OTG-INFO-####)• Configuration and Deployment Management Testing (OTG-CONFIG-####)• Identity Management Testing (OTG-IDENT-####)• Authentication Testing (OTG-AUTHN-####)• Authorization Testing (OTG-AUTHZ-####) | <ul style="list-style-type: none">• Session Management Testing (OTG-SESS-####)• Input Validation Testing (OTG-INPVAL-####)• Testing for Error Handling (OTG-ERR-####)• Testing for Weak Cryptography (OTG-CRYPST-####)• Business Logic Testing (OTG-BUSLOGIC-####)• Client Side Testing (OTG-CLIENT-####) |
|--|--|

An example on the use of OWASP references for security testing:



After deriving **security requirements** from use and misuse case modelling:

1. Password requirements must be aligned with the current standards for sufficient complexity.
2. Accounts must be locked out after five unsuccessful log in attempts.
3. Log in error messages must be generic.

OWASP Web Security Testing Guide

From **security requirements**, zoom in to appropriate category of OWASP ASVS to find a list of verification requirements for the security controls; e.g.

V2.1: Password Security Requirements:

1. Verify that user set passwords are at least 12 characters in length. (C6) (C6: Implement Digital Identity in [OWASP Top 10 Proactive Controls](#))
2. Verify that passwords 64 characters or longer are permitted. (C6)
3. Verify that passwords can contain spaces and truncation is not performed. Consecutive multiple spaces MAY optionally be coalesced. (C6)
4. Verify that Unicode characters are permitted in passwords. A single Unicode code point is considered a character, so 12 emoji or 64 kanji characters should be valid and permitted.
5. Verify that users can change their password.
6. Verify that password change functionality requires the user's current and new password.

C6: Implement Digital Identity in OWASP Top 10 Proactive Controls

Password Requirements

- Passwords should comply with the following requirements at the very least:
- **be at least 8 characters in length if multi-factor authentication (MFA) and other controls are also used. If MFA is not possible, this should be increased to at least 10 characters**
- all printing ASCII characters as well as the space character should be acceptable in memorized secrets
- encourage the use of long passwords and passphrases
- remove complexity requirements as these have been found to be of limited effectiveness. Instead, the adoption of MFA or longer password lengths is recommended
- ensure that passwords used are not commonly used passwords that have been already been leaked in a previous compromise. You may choose to block the top 1000 or 10000 most common passwords which meet the above length requirements and are found in compromised password lists. The following link contains the most commonly found passwords:
<https://github.com/danielmiessler/SecLists/tree/master/Passwords>

OWASP Web Security Testing Guide

In addition, refer to corresponding category of OWASP Testing Guide, e.g. Authentication Testing which provides a collection of related test cases as shown:

- Testing for Credentials Transported over an Encrypted Channel (OTG-AUTHN- 001)
- Testing for default credentials (OTG-AUTHN-002)
- Testing for Weak lock out mechanism (OTG-AUTHN-003)
- Testing for Bypassing Authentication Schema (OTG-AUTHN-004)
- Testing for Vulnerable Remember Password (OTG-AUTHN-005)
- Testing for Browser cache weakness (OTG-AUTHN-006)
- [Testing for Weak password policy](#) (OTG-AUTHN-007)
- Testing for Weak security question/answer (OTG-AUTHN-008)
- Testing for weak password change or reset functionalities (OTG-AUTHN-009)
- Testing for Weaker authentication in alternative channel (OTG-AUTHN-010)

OWASP Web Security Testing Guide

Testing for Weak or Unenforced Username Policy

ID
WSTG-IDNT-05

Summary

User account names are often highly structured (e.g. Joe Bloggs account name is jbloggs and Fred Nurks account name is fnurks) and valid account names can easily be guessed.

Test Objectives

- Determine whether a consistent account name structure renders the application vulnerable to account enumeration.
- Determine whether the application's error messages permit account enumeration.

How to Test

- Determine the structure of account names.
- Evaluate the application's response to valid and invalid account names.
- Use different responses to valid and invalid account names to enumerate valid account names.
- Use account name dictionaries to enumerate valid account names.

Remediation

Ensure the application returns consistent generic error messages in response to invalid account name, password or other user credentials entered during the log in process.

- Zoom in to relevant collection to find recommendations on how to test, e.g. Testing for weak password policy:

OWASP Web Security Testing Guide

Zoom in to relevant collection to find recommendations on how to test, e.g.

Testing for weak password policy:

Test Objectives:

- Determine the resistance of application against **brute force** password guessing using available **password dictionaries** by evaluating the **length**, **complexity**, **reuse** and **aging** requirements of passwords.

How to Test

1. What characters are permitted and forbidden for use within a password? Is the user required to use characters from different character sets such as lower and uppercase letters, digits and special symbols?
2. How often can a user change their password? How quickly can a user change their password after a previous change? Users may bypass password history requirements by changing their password 5 times in a row so that after the last password change they have configured their initial password again.
3. When must a user change their password?
 - Both NIST and NCSC recommend **against forcing regular password expiry**, although it may be required by standards such as PCI DSS.

OWASP Web Security Testing Guide

4. How often can a user reuse a password? Does the application **maintain a history** of the user's previous used 8 passwords?
5. **How different** must the next password be from the last password?
6. Is the user prevented from **using his username or other account information (such as first or last name) in the password**?
7. What are the minimum and maximum password lengths that can be set, and are they appropriate for the sensitivity of the account and application?
8. Is it possible set common passwords such as Password1 or 123456?

OWASP Web Security Testing Guide

- Next, for requirement 2, “Accounts must be to locked out after five unsuccessful log in attempts.” Then we have to take a look on the
- “Testing for Weak Lock Out Mechanism”

Summary

Account lockout mechanisms are used to mitigate brute force attacks. Some of the attacks that can be defeated by using lockout mechanism:

- Login password or username guessing attack.
- Code guessing on any 2FA functionality or Security Questions.

Account lockout mechanisms require a balance between protecting accounts from unauthorized access and protecting users from being denied authorized access. Accounts are typically locked after 3 to 5 unsuccessful attempts and can only be unlocked after a predetermined period of time, via a self-service unlock mechanism, or intervention by an administrator.

Despite it being easy to conduct brute force attacks, the result of a successful attack is dangerous as the attacker will have full access on the user account and with it all the functionality and services they have access to.

OWASP Web Security Testing Guide

Test Objectives

- Evaluate the account lockout mechanism's ability to mitigate brute force password guessing.
- Evaluate the unlock mechanism's resistance to unauthorized account unlocking.

OWASP Web Security Testing Guide

How to Test

- To test the strength of lockout mechanisms, you will need access to an account that **you are willing or can afford to lock**. If you have only one account with which you can log on to the web application, perform this test at the end of your test plan to avoid losing testing time by being locked out.
- To evaluate the account lockout mechanism's ability to **mitigate brute force password guessing**, attempt an invalid log in by using the incorrect password a number of times, before using the correct password to verify that the account was locked out.

Lockout Mechanism

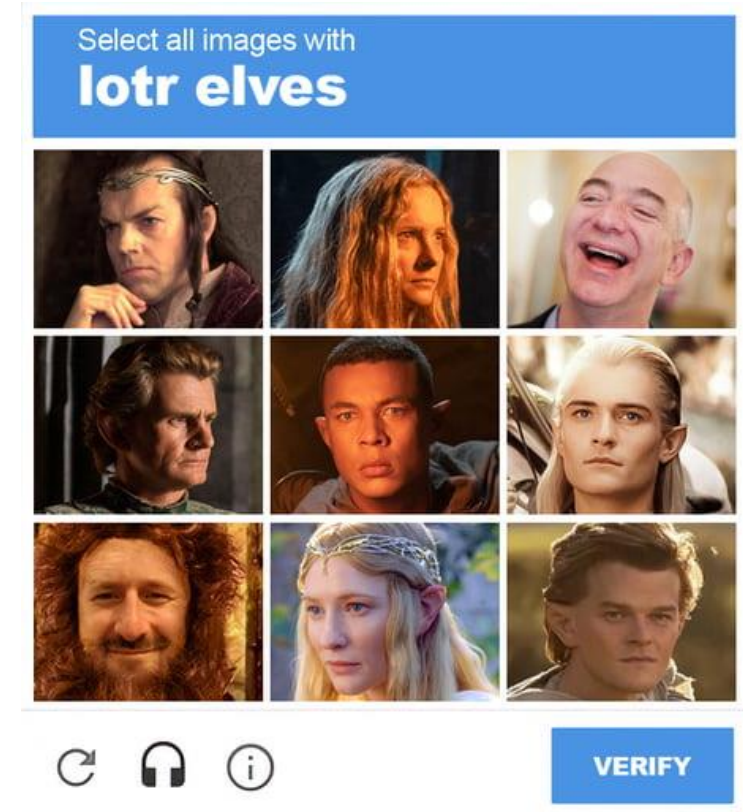
An example test may be as follows:

1. Attempt to log in with an incorrect password 3 times.
2. Successfully log in with the correct password, thereby showing that the lockout mechanism doesn't trigger after 3 incorrect authentication attempts.
3. Attempt to log in with an incorrect password 4 times.
4. Successfully log in with the correct password, thereby showing that the lockout mechanism doesn't trigger after 4 incorrect authentication attempts.
5. Attempt to log in with an incorrect password 5 times.
6. Attempt to log in with the correct password. The application returns "Your account is locked out.", thereby confirming that the account is locked out after 5 incorrect authentication attempts.
7. Attempt to log in with the correct password 5 minutes later. The application returns "Your account is locked out.", thereby showing that the lockout mechanism does not automatically unlock after 5 minutes.
8. Attempt to log in with the correct password 10 minutes later. The application returns "Your account is locked out.", thereby showing that the lockout mechanism does not automatically unlock after 10 minutes.
9. Successfully log in with the correct password 15 minutes later, thereby showing that the lockout mechanism automatically unlocks after a 10 to 15 minute period.

CAPTCHA

A CAPTCHA may hinder brute force attacks, but they can come with **their own set of weaknesses**, and should not replace a lockout mechanism. A CAPTCHA mechanism may be bypassed if implemented incorrectly. CAPTCHA flaws include:

- Easily defeated challenge, such as arithmetic or limited question set.
- CAPTCHA **checks for HTTP response code** instead of response success.
- CAPTCHA server-side logic defaults to a successful solve.
- CAPTCHA challenge result is never validated server-side.
- CAPTCHA input field or parameter is manually processed, and is improperly validated or escaped.



CAPTCHA

To evaluate CAPTCHA effectiveness:

1. Assess CAPTCHA challenges and attempt automating solutions depending on difficulty.
 2. Attempt to submit request without solving CAPTCHA via the normal UI mechanism(s).
 3. Attempt to submit request with intentional CAPTCHA challenge failure.
 4. Attempt to submit request without solving CAPTCHA (assuming some default values may be passed by client-side code, etc) while using a testing proxy (request submitted directly server-side).
 5. Attempt to fuzz CAPTCHA data entry points (if present) with common injection payloads or special characters sequences.
 6. Check if the solution to the CAPTCHA might be the alt-text of the image(s), filename(s), or a value in an associated hidden field.
 7. Attempt to re-submit previously identified known good responses.
 8. Check if clearing cookies causes the CAPTCHA to be bypassed (for example if the CAPTCHA is only shown after a number of failures).
 9. If the CAPTCHA is part of a multi-step process, attempt to simply access or complete a step beyond the CAPTCHA (for example if CAPTCHA is the first step in a login process, try simply submitting the second step [username and password]).
 10. Check for alternative methods that might not have CAPTCHA enforced, such as an API endpoint meant to facilitate mobile app access.
- Repeat this process to every possible functionality that could require a lockout mechanism.

Unlock Mechanism

To evaluate the unlock mechanism's resistance to unauthorized account unlocking, initiate the unlock mechanism and **look for weaknesses**. Typical unlock mechanisms may involve secret questions or an emailed unlock link. The unlock link should be a unique one-time link, to stop an attacker from guessing or replaying the link and performing brute force attacks in batches.

Note that an unlock mechanism should only be used for unlocking accounts. It is not the same as a password recovery mechanism, yet could follow the same security practices.

Remediation

Apply account unlock mechanisms depending on the risk level. In order from lowest to highest assurance:

1. Time-based lockout and unlock.
2. Self-service unlock (sends unlock email to registered email address).
3. Manual administrator unlock.
4. Manual administrator unlock with positive user identification.

OWASP Web Security Testing Guide

Last, for **security requirement** 3: Log in error messages must be generic, refer Testing for Account Enumeration and Guessable User Account:

How to Test

- Testing for valid user with wrong password
 - The browser should display a message similar to the following one:

Authentication failed.

[Return to Login page](#)

- Or something like:

No configuration found.

Contact your system administrator.

[Return to Login page](#)

- Against any message that reveals the existence of user, for instance, similar to:

Login for User foo: invalid password

- ...

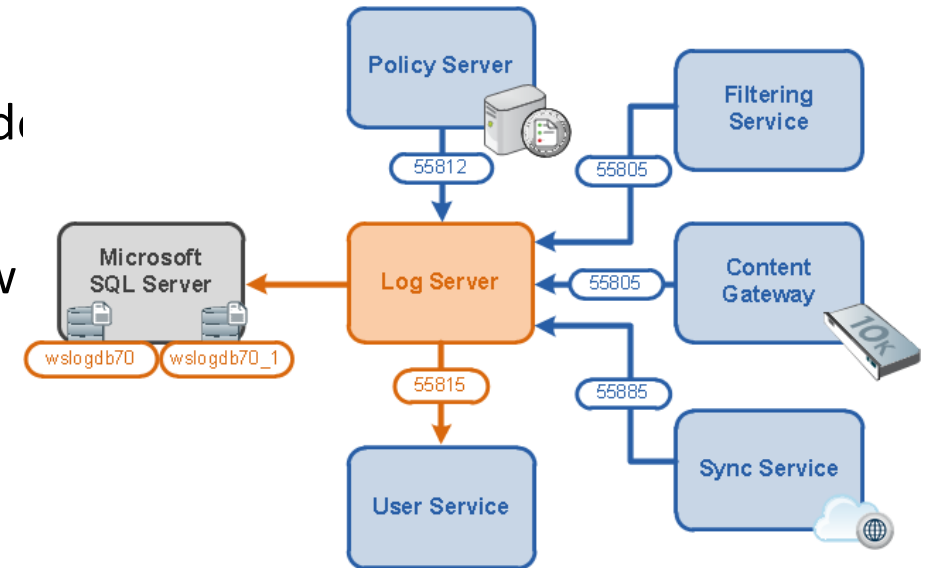
More Logging considerations

Event logs will often contain data that is useful to an attacker (information leakage) or can be used directly in exploits:

- Debug information
- Stack traces
- Usernames
- System component names
- Internal IP addresses
- Less sensitive personal data (e.g. email addresses, postal addresses and telephone numbers associated with
- named individuals)
- Business data

Log Location

- Servers will generate local logs of their actions and errors, consuming the disk of the system the server is running on.
- If the server is compromised its logs can be wiped out by the intruder to **clean up all the traces of its attack and methods**. If this were to happen the system administrator would have no knowledge of how the attack occurred or where the attack source was located.
- Most attacker tool kits include a “log zapper” that is capable of cleaning up any logs that hold given information (like the IP address of the attacker) and are routinely used in attacker’s system-level root kits.
- Consequently, it is wiser to **keep logs in a separate location** and not in the web server itself.



Log Rotation

Most servers (but few custom applications) will rotate logs in order to prevent them from filling up the file system they reside on. The assumption when rotating logs is that the information in them is only necessary for a limited amount of time.

This feature should be tested in order to ensure that:

- Logs are **kept for the time defined** in the security policy, not more and not less.
- Logs are compressed once rotated (this is a convenience, since it will mean that more logs will be stored for the same available disk space).
- File system permission of rotated log files are the same (or stricter) than those of the log files themselves. For example, web servers will need to write to the logs they use but they don't actually need to write to rotated logs, which means that the permissions of the files can be changed upon rotation to prevent the web server process from modifying these.
- Some servers might **rotate logs when they reach a given size**. If this happens, it must be ensured that an attacker **cannot force logs to rotate** in order to hide his tracks.

Log Access Control

- Event log information **should never be visible to end users**. Even **web administrators** should not be able to see such logs since it breaks separation of duty controls.
- Ensure that any access control schema that is used to protect access to **raw logs and any applications providing capabilities to view or search the logs** is not linked with access control schemas for other application user roles.
- Neither should any log data be viewable by unauthenticated users.

Log Review

Review of logs can be used for more than extraction of usage statistics of files in the web servers (which is typically what most log-based application will focus on), but also to determine if attacks take place at the web server.

In order to analyze web server attacks the error log files of the server need to be analyzed. Review should concentrate on:

- **40x (not found)** error messages. A large amount of these from the same source might be indicative of a CGI scanner tool being used against the web server
- **50x (server error)** messages. These can be an indication of an attacker abusing parts of the application which fail unexpectedly. For example, the first phases of a SQL injection attack will produce these error message when the SQL query is not properly constructed and its execution fails on the back-end database.

OWASP Web Security Testing Guide

Finally, after performing security tests, how to report?

<https://owasp.org/www-project-web-security-testing-guide/stable/5-Reporting/README>

A report should be easy to understand and should highlight all the risks found.

- Broadly, the report should contain:
 1. Introduction
 2. Executive Summary
 3. Findings Appendices
- e.g. a checklist of all the tests conducted, such as the [WSTG Checklist](#)

Note: Security test report is not required for D2 which is on implementation, but you may include a checklist for automated tests performed in D2.

Deliverable 3 details

1. Implementation walkthrough

- Based on D2 implementation report from the development team (to be assigned)
- Feedback/comment whether sufficient explanation of the logic and flow of codes to commence secure software verification by your QA/security team
- If no, elaborate what should be included
- **No communication/argument between development team and QA/security team**

2. Source code review

- Analyze manually the assigned development team's codes
- Describe the findings
- Suggest if there are anything is missing, and if there are any vulnerabilities, the way to fix them.

3. Static code analysis

- Analyze using suitable automated tool separately **(1) your own codes + (2) the assigned development team's codes**
- Describe the findings
- If there are any vulnerabilities, to include the way to fix them. (but do **NOT** fix them if it is your own code)

Deliverable 3 details

4. Dynamic App Security Testing (DAST)

- Analyze dynamically the assigned development team's codes
- Describe the findings
- If there are any vulnerabilities, to include the way to fix them.

5. Overall

- Provide solutions, with references to solve the vulnerabilities. (include CVE and CWE are bonus).
- A conclusion section to summarise the findings, comments about the application. Rate the application in terms of the security level (1 min, 5 max) and usability level (1 min, 5 max).
- Justify the rating.
- **Page Limit: 35 pages (excluding the appendices: marker does not need to read)**
- **Deadline: Week 13 Monday 27 Jul 9:00 am**

Rebuttal Phase (Optional)

- Dev team will receive the report of QA team on 27 Jul
- Dev team prepare a rebuttal report (maximum 1 page)
- Submission deadline: **Week 13 Thursday 30 Jul 9:00 am**

QUESTION AND COMMENTS?

Reference

- <https://owasp.org/www-project-application-security-verification-standard/>